Machine Learning e Pattern Recognition – Exam Collection

Mellow

29 giugno 2026

Indice

1 PCA

Motivation. Dimensionality reduction maps data from an n-dimensional space to an m-dimensional one, with $m \ll n$. The practical purposes are four-fold: **to compress information, remove unwanted variability (noise), simplify classification by reducing the number of parameters to estimate, and visualize the data.**

PCA is the standard linear and unsupervised technique for this task. A sample is projected to the low-dimensional space as $\mathbf{y} = P^T\mathbf{x}$ and reconstructed in the original space as $\hat{\mathbf{x}} = P\mathbf{y} = PP^T\mathbf{x}$. The matrix $P \in \mathbb{R}^{n\times m}$ with orthonormal columns minimizes the average reconstruction error:

$$P^* = \arg\min_P \frac{1}{K}\sum_i \|\mathbf{x}_i - PP^T\mathbf{x}_i\|^2 \quad \text{s.t. } P^TP = I$$

Equivalently, it maximizes the projected variance:

$$\widetilde{L}(P) = \mathrm{Tr}\left(P^T \underbrace{\frac{1}{K}\sum_i \mathbf{x}_i\mathbf{x}_i^T}_{=C} P \right)$$

The solution consists of the m eigenvectors of C with the largest eigenvalues. From the decomposition $C = U\Sigma U^T$:

$$P^* = [\mathbf{u}_1, \ldots, \mathbf{u}_m]$$

The result is a coordinate system where each axis captures the maximum remaining variance, and the variance along each axis corresponds to the associated eigenvalue.

Non-centered data. If the data are not zero-mean, the first eigenvector points towards the dataset mean — not informative. The mean $\bar{\mathbf{x}}$ should always be subtracted, and C becomes the empirical covariance matrix:

$$C = \frac{1}{K}\sum_i (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T$$

Choice of m. m is a hyperparameter: it does not emerge from the PCA solution itself. Two approaches: cross-validation, or a criterion on explained variance — choose the smallest m such that:

$$\frac{\sum_{i=1}^{m} \sigma_i}{\sum_{i=1}^{n} \sigma_i} \geq t \quad \text{(e.g. } t = 0.95\text{)}$$

The choice of m introduces a trade-off. If m is too large, too many dimensions remain, increasing the number of parameters to estimate in the downstream classifier and the risk of overfitting. If m is too small, discriminant directions may be lost, causing underfitting. Cross-validation allows balancing this trade-off for the specific task.

Limitations. Computing C is expensive in high-dimensional spaces. Most importantly: being unsupervised, PCA does not guarantee to preserve discriminant directions — directions of highest variance do not necessarily coincide with those useful for class separation. **A further practical advantage is that PCA automatically eliminates directions with zero or very low variance, which could cause numerical instability in some classifiers** (e.g., division by zero variance).

2 LDA

Motivation. LDA is a linear **supervised** technique for dimensionality reduction and classification. **Unlike PCA, which maximizes variance without knowledge of class labels, LDA exploits class information to find directions that explicitly separate groups.** The goal is to find the direction $\mathbf{w}$ that maximizes the separation between classes relative to the within-class scatter:

$$\max_{\mathbf{w}} L(\mathbf{w}) = \frac{s_B}{s_W} = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}$$

where the scatter matrices are:

$$S_B = \frac{1}{N} \sum_{c=1}^{K} n_c (\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^T, \qquad S_W = \frac{1}{N} \sum_{c=1}^{K} \sum_{i=1}^{n_c} (\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)(\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)^T$$

S_B measures how far the class means are from the global mean; S_W measures how spread out the samples are around their class mean. Maximizing the ratio means finding a projection where classes are compact internally and far apart from each other.

Solution derivation. Setting $\nabla_{\mathbf{w}} L = 0$, dividing by the scalar $(\mathbf{w}^T S_W \mathbf{w})$ and multiplying by S_W^{-1}, leads to the eigenvalue problem:

$$S_W^{-1} S_B \mathbf{w} = \lambda(\mathbf{w})\, \mathbf{w}, \qquad \lambda(\mathbf{w}) = L(\mathbf{w})$$

Maximizing L is thus equivalent to maximizing λ: the optimal direction is the eigenvector of $S_W^{-1} S_B$ with the largest eigenvalue.

Binary case. With two classes, $S_B = k(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^T$, which has rank 1, and the solution simplifies in closed form:

$$\mathbf{w} \propto S_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)$$

This formula has a direct interpretation: compute the difference between the two class means and "correct" it for within-class scatter via S_W^{-1}. If the classes have the same within-class scatter ($S_W = I$), the optimal direction is simply the one joining the two means.

The classification rule assigns $\mathbf{x}_t$ to the closest class in the projected space, with threshold $t = \frac{m_1 + m_2}{2}$:

$$C(\mathbf{x}_t) = \begin{cases} C_1 & \mathbf{w}^T \mathbf{x}_t < t \\ C_2 & \mathbf{w}^T \mathbf{x}_t \geq t \end{cases}$$

The threshold t is the mean of the projected class means: it is the optimal choice under the assumption of equal within-class variance and equal prior. If the priors differ, the threshold must be shifted accordingly.

Extension to m dimensions. The m most discriminant directions are collected in $W \in \mathbb{R}^{n \times m}$. The criterion becomes:

$$\max_W \ \mathrm{Tr}\left(\tilde{S}_W^{-1} \tilde{S}_B\right), \quad \tilde{S}_B = W^T S_B W,\ \tilde{S}_W = W^T S_W W$$

The solution consists of the m eigenvectors of $S_W^{-1}S_B$ with the largest eigenvalues. **An important structural constraint: the rank of S_B is at most $K-1$, because it is defined by K means constrained to sum to the global mean. As a result, the maximum number of discriminant directions that can be extracted is $K-1$, regardless of the original data dimension. For binary classification, there is always only one direction.**

Limitations and practical considerations. LDA assumes that the within-class distributions are Gaussian with the same covariance matrix — an assumption often violated in practice. S_W can become singular when the number of samples is less than the data dimension, making S_W^{-1} not computable. **A common solution is to apply PCA as a pre-processing step before LDA: PCA reduces dimensionality by eliminating directions with zero or near-zero variance, making S_W invertible and reducing the risk of overfitting. However, care must be taken not to eliminate discriminant directions with PCA before LDA can find them — the number of PCA components should be chosen carefully, typically via cross-validation.**

3 PCA vs LDA

Both **PCA** and **LDA** are linear dimensionality reduction techniques that map data from an n-dimensional feature space to a lower m-dimensional space ($m \ll n$), and both address the problem from a geometric point of view, but they differ fundamentally in supervision, objective, and the nature of the directions they find.

Goals and Formulation

PCA is an *unsupervised* technique. Its goal is to find an orthogonal projection matrix $P \in \mathbb{R}^{n\times m}$ that best preserves the structure of the data, without any knowledge of class labels. Data is first centered by subtracting the mean $\bar{\mathbf{x}}$, and then PCA seeks directions that capture the most variance. The projection of a sample is $\mathbf{y} = P^T\mathbf{x}$, and the reconstruction in the original space is $\hat{\mathbf{x}} = P\mathbf{y}$.

LDA is a *supervised* technique. Its goal is to find a projection matrix $W \in \mathbb{R}^{n \times m}$ such that the projected data has maximum separation between classes and minimum spread within each class. It explicitly uses class labels to compute class means $\boldsymbol{\mu}_c$ and the global mean $\boldsymbol{\mu}$. The projection of a sample is $\tilde{\mathbf{x}} = W^T \mathbf{x}$.

Training Objective

The PCA objective is to **minimize the average reconstruction error**:

$$P^* = \arg\min_P \frac{1}{K} \sum_{i=1}^{K} \|\mathbf{x}_i - PP^T \mathbf{x}_i\|^2, \quad \text{s.t. } P^T P = I$$

This is equivalent to **maximizing the projected variance**:

$$\widetilde{L}(P) = \mathrm{Tr}\big(P^T C\, P\big)$$

where $C = \frac{1}{K} \sum_i (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T$ is the empirical covariance matrix. The constraint $P^T P = I$ ensures orthonormality of the components.

The LDA objective is to **maximize the Fisher ratio** — the ratio of between-class variability to within-class variability. **For the multi-direction case, the criterion is:**

$$L = \mathrm{Tr}\Big(\widetilde{S}_W^{-1} \widetilde{S}_B\Big) = \mathrm{Tr}\big((W^T S_W W)^{-1} (W^T S_B W)\big)$$

where the scatter matrices are:

$$S_B = \frac{1}{N} \sum_{c=1}^{K} n_c (\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^T, \quad S_W = \frac{1}{N} \sum_{c=1}^{K} \sum_{i=1}^{n_c} (\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)(\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)^T$$

Unlike PCA, LDA does *not* require W to be orthogonal.

Characteristics of the Directions

The table below summarizes the key properties of the directions found by each method:

Property	PCA Principal Components	LDA Discriminant Directions
Computed from	Eigenvectors of covariance matrix C	Eigenvectors of $S_W^{-1}S_B$
Ordering	By descending eigenvalue (= variance)	By descending eigenvalue (= Fisher ratio)
Orthogonality	Guaranteed ($P^TP = I$)	Not required
Max number of directions	Up to n (feature dimensionality)	At most $K-1$ (number of classes minus one)
Label-awareness	No — may not be discriminant	Yes — explicitly maximizes class separation
Interpretability	Directions of highest variance	Directions of highest class separability

The $K-1$ limit on LDA directions follows directly from the rank of S_B: since S_B is a sum of K rank-one matrices centered around the global mean, it has at most $K-1$ non-zero eigenvalues. In contrast, PCA can extract up to n components, selected by retaining enough to explain a target fraction t of total variance:

$$\min_m \; m \quad \text{s.t.} \quad \frac{\sum_{i=1}^{m} \sigma_i}{\sum_{i=1}^{n} \sigma_i} \geq t$$

Use in Classification

PCA for classification is used as a *preprocessing step*. It reduces dimensionality before feeding data into a separate classifier (e.g., logistic regression, k-NN). Because PCA is unsupervised, there is no guarantee the retained components are discriminant — variance-maximizing directions and class-separating directions can differ significantly.

LDA for classification works directly. For a binary problem, the optimal direction is:

$$\mathbf{w} \propto S_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)$$

and the classification rule assigns sample $\mathbf{x}_t$ to class C_1 if $\mathbf{w}^T\mathbf{x}_t < t$, where the threshold is the midpoint of the projected class means:

$$t = \frac{m_1 + m_2}{2}, \quad m_c = \mathbf{w}^T\boldsymbol{\mu}_c$$

This is equivalent to a nearest-centroid classifier in the projected space. LDA implicitly assumes that within-class distributions are Gaussian with equal covariance, so the linear decision boundary is optimal under those assumptions.

A common practical strategy is to **apply PCA first, then LDA**: PCA removes noise and reduces the risk of S_W becoming singular (which happens

when the number of features is large relative to the number of samples), and LDA then finds the most discriminant subspace within the PCA-reduced space.

4 Generative Gaussian Models

Motivation. Generative Gaussian Models (GGM) address closed-set classification by explicitly modeling the data-generation process for each class. **Unlike discriminative models that learn decision boundaries directly, generative models assume a probabilistic class-conditional density and apply Bayes' theorem to compute posterior probabilities.** The fundamental assumption is that the likelihood of the data given a class follows a multivariate Gaussian distribution, allowing us to exploit the well-understood properties of Gaussians to derive tractable classification rules.

Probabilistic framework. Classification is performed by computing the posterior probability:

$$P(C_t = c|X_t = \mathbf{x}_t) = \frac{f_{X|C}(\mathbf{x}_t|c) \cdot P(C = c)}{\sum_{c'} f_{X|C}(\mathbf{x}_t|c') \cdot P(C = c')}$$

The classifier assigns $\mathbf{x}_t$ to the class with the highest posterior. The joint density factorizes as:

$$f_{X,C}(\mathbf{x}, c) = f_{X|C}(\mathbf{x}|c) \cdot P(C = c)$$

The class prior $P(C = c)$ is determined by the application; the class likelihood $f_{X|C}(\mathbf{x}|c)$ is learned from data.

Mathematical formulation. GGM assumes that each class likelihood is a multivariate Gaussian:

$$(X_t|C_t = c) \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$$

The log-likelihood per sample in class c is:

$$\log f_{X|C}(\mathbf{x}|c) = -\frac{D}{2} \log 2\pi - \frac{1}{2} \log |\boldsymbol{\Sigma}_c| - \frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_c)^T \boldsymbol{\Sigma}_c^{-1}(\mathbf{x} - \boldsymbol{\mu}_c)$$

Training via ML estimation. Parameters $\boldsymbol{\mu}_c$ and $\boldsymbol{\Sigma}_c$ are estimated by maximizing the log-likelihood class by class. The closed-form ML solutions are:

$$\boldsymbol{\mu}_c^* = \frac{1}{N_c} \sum_{i:c_i=c} \mathbf{x}_i, \qquad \boldsymbol{\Sigma}_c^* = \frac{1}{N_c} \sum_{i:c_i=c} (\mathbf{x}_i - \boldsymbol{\mu}_c^*)(\mathbf{x}_i - \boldsymbol{\mu}_c^*)^T$$

where N_c is the number of samples in class c. **Each class is estimated independently: the sample mean is the class center, and the sample covariance captures the within-class spread.**

Inference: binary classification and LLR. For a binary decision, the classifier compares the log-likelihood ratio (LLR) to a threshold derived from the log-odds:

$$\text{llr}(\mathbf{x}_t) = \log \frac{f_{X|C}(\mathbf{x}_t|c_1)}{f_{X|C}(\mathbf{x}_t|c_0)} \gtrless -\log \frac{P(C=c_1)}{P(C=c_0)}$$

The LLR is a **quadratic function of x:**

$$\text{llr}(\mathbf{x}) = \mathbf{x}^T A \mathbf{x} + \mathbf{x}^T \mathbf{b} + c$$

where the quadratic term $A = -\frac{1}{2}(\boldsymbol{\Lambda}_1 - \boldsymbol{\Lambda}_0)$ vanishes only when the two classes share the same covariance ($\boldsymbol{\Sigma}_1 = \boldsymbol{\Sigma}_0$). This produces a **quadratic decision boundary,** characteristic of QDA (Quadratic Discriminant Analysis).

Inference: multiclass classification. For $K > 2$ classes, assign the sample to:

$$c_t^* = \arg\max_h \left[\log f_{X|C}(\mathbf{x}_t|h) + \log P(C=h)\right]$$

The classifier outputs class-conditional log-likelihoods; the prior term $\log P(C = h)$ is application-dependent and can be adjusted at deployment time.

Model variants. GGM encompasses several important variants:

- **Full covariance (QDA):** Each class has its own full covariance matrix $\boldsymbol{\Sigma}_c$. Highly flexible but requires many samples; decision boundary is quadratic.

- **Tied covariance (LDA-like):** All classes share a common covariance $\boldsymbol{\Sigma}$:

$$\boldsymbol{\Sigma}^* = \frac{1}{N} \sum_c \sum_{i:c_i=c} (\mathbf{x}_i - \boldsymbol{\mu}_c^*)(\mathbf{x}_i - \boldsymbol{\mu}_c^*)^T$$

  This reduces the number of parameters and produces a **linear decision boundary** (the quadratic term A cancels). More stable with limited data.

- **Naive Bayes Gaussian:** Each class has a diagonal covariance, assuming feature independence. Simple and computationally efficient, but assumes uncorrelated features.

Limitations. GGM assumes all data within a class are drawn from a Gaussian distribution — a strong assumption often violated in practice. **Estimating a full covariance matrix requires $\frac{D(D+1)}{2}$ parameters,** which becomes prohibitive in high dimensions and with limited data. The covariance matrix must be invertible; if $N < D$, standard ML estimation fails. A common solution is to apply PCA beforehand to reduce dimensionality and ensure invertibility, though care must be taken not to eliminate discriminant directions.

5 LDA vs Tied MVG

Both **LDA** (binary) and the **Tied-covariance MVG** classifier produce a **linear decision boundary**, and in fact coincide under equal priors. They reach the same rule from two different criteria: LDA from a *geometric* objective (Fisher ratio), Tied MVG from a *generative probabilistic* model.

Model Formulation, Training and Inference

LDA classifier. LDA maximizes the **Fisher ratio**, the ratio of between-class to within-class scatter:

$$\max_{\mathbf{w}} \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}, \qquad S_B = \tfrac{1}{N}\sum_c n_c(\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^T, \;\; S_W = \tfrac{1}{N}\sum_c \sum_i (\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)(\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)^T.$$

For the binary case the solution is in closed form, $\mathbf{w} \propto S_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)$, and inference projects the sample as $\mathbf{w}^T\mathbf{x}_t$.

Tied MVG classifier. Each class likelihood is Gaussian with a **single shared covariance**, $(X \mid C = c) \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma})$. Parameters are estimated by ML: the means are the class sample means, and $\boldsymbol{\Sigma}$ is the **pooled within-class covariance**

$$\boldsymbol{\Sigma}^* = \frac{1}{N} \sum_c \sum_{i:c_i=c} (\mathbf{x}_i - \boldsymbol{\mu}_c^*)(\mathbf{x}_i - \boldsymbol{\mu}_c^*)^T.$$

Inference is a Bayes decision: compare the log-likelihood ratio to the log-odds threshold.

Model assumptions. Both models make the *same* assumption: within-class data are Gaussian with a **common (tied) covariance**. The key structural identity is that LDA's within-class scatter S_W **is** the tied ML covariance $\boldsymbol{\Sigma}$ (up to the $1/N$ normalization).

The Relationship and the Decision Rules

For tied covariance the quadratic term of the MVG log-likelihood ratio, $A = -\frac{1}{2}(\boldsymbol{\Sigma}_1^{-1} - \boldsymbol{\Sigma}_0^{-1})$, **vanishes**, leaving a purely linear score with direction $\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)$ — the **same direction** as LDA's $S_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)$. Setting the LLR to zero (equal priors) gives

$$(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)^T\boldsymbol{\Sigma}^{-1}\mathbf{x} = \tfrac{1}{2}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)^T\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 + \boldsymbol{\mu}_0) = \frac{m_1 + m_0}{2},$$

which is exactly LDA's midpoint threshold. **Under equal priors the two binary classifiers are identical** — same direction *and* same threshold — both reducing to a nearest-centroid (Mahalanobis) rule in the $\boldsymbol{\Sigma}$-whitened space.

The decision rules differ only in how the threshold is set:

Aspect	**LDA (binary)**	**Tied MVG (binary)**
Objective	Maximize Fisher ratio (geometric)	Maximize per-class Gaussian likelihood (generative)
Score direction	$S_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)$	$\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)$ — same since $\boldsymbol{\Sigma} \propto S_W$
Decision rule	$\mathbf{w}^T\mathbf{x}_t < t,\ \ t = \frac{m_1+m_2}{2}$	$\mathrm{llr}(\mathbf{x}_t) \gtrless -\log \frac{P(c_1)}{P(c_0)}$
Threshold / priors	Midpoint; optimal only for equal priors	Log-odds; natively handles priors and costs
Boundary	Linear	Linear (quadratic term A cancels)

The **practical difference** is that the Tied MVG threshold is derived from the application priors (and can be shifted for costs), whereas LDA's midpoint is optimal only when the priors are equal — if they differ, LDA's threshold must be adjusted by hand.

LDA for Multiclass Dimensionality Reduction

For multiclass problems LDA is used as a **dimensionality-reduction** technique: it collects the m most discriminant directions in W, maximizing

$$\max_{W} \operatorname{Tr}\big((W^T S_W W)^{-1}(W^T S_B W)\big)\,.$$

Limitations. Since S_B has rank at most $K-1$, LDA can extract **at most** $K-1$ **directions** regardless of the feature dimension — a hard ceiling that discards potentially useful directions when K is small. It also assumes equal-covariance Gaussian classes (often violated), and S_W becomes singular when $N < D$ (a common remedy is applying PCA first). In contrast, the Tied MVG handles the multiclass case *directly* via per-class posteriors, with no $K-1$ ceiling.

6 Logistic Regression

Logistic Regression is a **discriminative** model for binary classification: instead of modelling the class-conditional densities, it models the **class posterior** $P(C \mid X)$ directly. It assumes the log-posterior-ratio is a **linear** function of the input, so the posterior is a sigmoid of a linear score.

Classification Rule and Probabilistic Interpretation

Assuming a linear decision rule parametrized by $(\mathbf{w}, b)$, the posterior for class h_1 is the **sigmoid** of a linear score:

$$P(C{=}h_1 \mid \mathbf{x}, \mathbf{w}, b) = \frac{1}{1+e^{-(\mathbf{w}^T\mathbf{x}+b)}} = \sigma(\mathbf{w}^T\mathbf{x}+b), \qquad \sigma(t) = \frac{1}{1+e^{-t}}.$$

The **score** $s = \mathbf{w}^T\mathbf{x} + b$ has a precise probabilistic meaning: it is the **log-posterior-ratio**

$$s = \mathbf{w}^T\mathbf{x} + b = \log \frac{P(C{=}1 \,|\, \mathbf{x})}{P(C{=}0 \,|\, \mathbf{x})}.$$

The **classification rule** is therefore $s \gtrless 0$: a **linear hyperplane orthogonal to $\mathbf{w}$**. The magnitude $|s|$ relates to the (signed) distance from the separating surface, i.e. how confident the prediction is. Being discriminative, the model does *not* need an explicit model of the feature density $f_X(\mathbf{x})$: in the joint likelihood $f_{X,C} = P(C \,|\, X, \theta)\, f_X(\mathbf{x})$ the term f_X does not depend on $\theta = (\mathbf{w}, b)$ and is dropped.

Parameter Estimation and the Training Objective

Parameters are estimated by **Maximum Likelihood** on the labels (i.i.d. samples). Writing $y_i = \sigma(\mathbf{w}^T\mathbf{x}_i + b)$, each label is Bernoulli, $C_i \,|\, \mathbf{x}_i \sim \mathrm{Ber}(y_i)$, so

$$\ell(\mathbf{w}, b) = \sum_{i=1}^{n} \big[c_i \log y_i + (1 - c_i) \log(1 - y_i)\big].$$

Maximizing ℓ is equivalent to minimizing $J = -\ell$, which is the average **cross-entropy** $H(P, Q) = \mathbb{E}_P[-\log Q]$ between the empirical label distribution $P = \mathrm{Ber}(c_i)$ and the model's prediction $Q = \mathrm{Ber}(y_i)$; it is minimized when $Q = P$.

Recasting the labels as $z_i = 2c_i - 1 \in \{-1, +1\}$ and writing $s_i = \mathbf{w}^T\mathbf{x}_i + b$, the same objective takes the explicit form of a sum of per-sample losses:

$$J(\mathbf{w}, b) = \sum_{i=1}^{n} \log\big(1 + e^{-z_i s_i}\big) = \sum_{i=1}^{n} \ell(-z_i s_i), \qquad \ell(t) = \log(1 + e^{-t}).$$

Here $z_i s_i$ is the cost of a linear prediction: when prediction and label agree ($z_i s_i > 0$) the cost decays (asymptotically) to 0 as $|s_i|$ grows; when they disagree it grows (asymptotically) *linearly* in $|s_i|$. This casts LR as **Empirical Risk Minimization (ERM)** $R(\theta) = \sum_i \ell(\theta, \mathbf{x}_i, z_i)$. The solution has **no closed form** and is found with a numerical solver from the loss and its gradient.

Regularization

If the classes are **linearly separable**, the loss has *no minimum*: it can be driven to its infimum $\inf J = 0$ by sending $\|\mathbf{w}\| \to \infty$, so the parameters do not converge. We add a **norm penalty** (regularization term), obtaining **regularized LR**:

$$R(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n} \log\bigl(1 + e^{-z_i s_i}\bigr),$$

an instance of **regularized risk minimization** $R = \Phi(\mathbf{w}) + \frac{1}{n}\sum_i \ell$. The hyper-parameter λ trades separation against simplicity (too large $\Rightarrow$ small norm but poor separation; too small $\Rightarrow$ overfitting) and is selected by **cross-validation** (it cannot be tuned by minimizing R, which would give $\lambda = 0$). Unlike the unregularized model, regularized LR is **not invariant** to linear transformations of the features, so pre-processing (centering, variance standardization, whitening, length normalization) can help.

Score, Priors and Recalibration

Since the parameters are trained on the labels, the score reflects the **empirical training prior**. To use it as a **log-likelihood ratio** for an arbitrary application, subtract the empirical prior log-odds:

$$s_{llr} = \mathbf{w}^T\mathbf{x} + b - \log\frac{n_T}{n_F}, \qquad \text{decide } s_{llr} \gtrless \log\frac{\pi_T}{1-\pi_T}.$$

If the application prior π_T is known *before* training, one can directly optimize **prior-weighted LR**, which reweights the two class losses by π_T/n_T and $(1-\pi_T)/n_F$; standard LR is the special case with the empirical prior.

Non-linear Decision Functions and the Multi-class Case

Non-linearity. LR can be trained on an **expanded feature space** $\phi(\mathbf{x})$: a linear rule $\mathbf{w}^T\phi(\mathbf{x}) + b$ in the expanded space corresponds to a **non-linear** (e.g. quadratic) boundary in the original space. The cost is that $\dim\phi$ can grow very quickly, raising computation and overfitting risk.

Multiclass (softmax). For K classes the posteriors take the **softmax** form

$$P(C{=}k \,|\, \mathbf{x}) = \frac{e^{\mathbf{w}_k^T \mathbf{x} + b_k}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^T \mathbf{x} + b_j}},$$

and training minimizes the multiclass cross-entropy (**softmax loss**) $-\sum_i \sum_k z_{ik} \log y_{ik}$ with 1-of-K labels. The model is **over-parametrized** (adding a constant vector to all $\mathbf{w}_j$ leaves it unchanged); a regularization term is added as in the binary case.

7 Logistic Regression vs Generative Gaussian Models

Logistic Regression (LR) and the **Generative Gaussian Models** (MVG) are the prototypical **discriminative** and **generative** classifiers. They reach a probabilistic decision in opposite ways, yet share the same family of decision functions.

What Each Model Estimates

The **generative** MVG models the *joint* distribution through the class-conditional densities and the prior, $f(\mathbf{x} \,|\, c)\, P(c)$, and obtains the posterior by Bayes. The **discriminative** LR models the posterior $P(c \,|\, \mathbf{x})$ *directly*, and does **not** need a model of the feature density $f_X(\mathbf{x})$. Consequently MVG is estimated in **closed form** (per-class ML means and covariances), whereas LR has **no closed form** and minimizes the cross-entropy / empirical risk numerically.

The Shared Linear Form (LR $\leftrightarrow$ Tied MVG)

The key connection is that the **Tied MVG** has a **linear** log-posterior-ratio

$$s = \mathbf{w}^T \mathbf{x} + b, \qquad \mathbf{w} \propto \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0),$$

which is *exactly* the parametric form LR assumes a priori. So **LR (with linear features) and the Tied MVG share the same hypothesis class**

— linear log-odds — and differ only in *how* they pick $(\mathbf{w}, b)$: MVG **generatively** (Gaussian ML, then derive the posterior), LR **discriminatively** (optimize the conditional likelihood, ignoring the densities). Likewise, the full MVG (QDA) gives a quadratic boundary, matched by LR trained on a quadratic feature expansion $\phi(\mathbf{x})$.

Assumptions, Priors and Trade-offs

MVG assumes **Gaussian** class-conditionals (a strong assumption): when it holds, MVG is more **data-efficient**; when it is violated, LR — which only assumes a linear log-odds — is often more robust. On **priors**, the generative model separates likelihood from prior, so priors and costs can be changed at deployment by moving the threshold (the LLR is Bayes-ready); LR instead embeds the **empirical training prior** in its score and requires **recalibration** ($s_{llr} = s - \log \frac{n_T}{n_F}$) or **prior-weighted** training. Finally, a full MVG needs $\frac{D(D+1)}{2}$ covariance parameters per class (hence $N > D$, often PCA first), while linear LR needs only $D + 1$ and scales better, at the price of a linear boundary unless features are expanded.

Aspect	**Logistic Regression**	**Generative Gaussian (MVG / Tied)**
Type	Discriminative: models $P(C \mid \mathbf{x})$ directly	Generative: models $f(\mathbf{x} \mid c)P(c)$, then Bayes
Training	ML / cross-entropy, numerical (no closed form)	ML in closed form, per class
Assumptions	Linear log-odds; no model of f_X	Gaussian class-conditionals
Decision rule	$s \gtrless 0$; linear (quadratic if $\phi(\mathbf{x})$)	$\text{llr} \gtrless -\log \frac{P(c_1)}{P(c_0)}$; quadratic (QDA), linear if tied
Priors	Score tied to empirical prior $\Rightarrow$ recalibrate	Prior separated $\Rightarrow$ threshold set natively
Parameters	$D + 1$ (linear)	full $\frac{D(D+1)}{2}$ per class; tied: one $\mathbf{\Sigma}$
Strength	Robust to non-Gaussian features; few parameters	Data-efficient if Gaussian holds; LLR Bayes-ready

8 Logistic Regression vs LDA

LDA (binary) and **Logistic Regression** (LR) both produce a **linear** binary classifier — a decision based on $s = \mathbf{w}^T\mathbf{x} + b$ with a boundary that is a hyperplane orthogonal to $\mathbf{w}$ — but they choose the separating rule from **different criteria**: LDA from a geometric objective, LR from a discriminative (probabilistic) one.

Objective and Training

LDA maximizes the **Fisher ratio**, the ratio of between- to within-class scatter,

$$\max_{\mathbf{w}} \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}, \qquad \mathbf{w} \propto S_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1),$$

a purely **geometric** criterion solved in **closed form** (a generalized eigenproblem). **LR** instead minimizes the **cross-entropy / empirical risk**

$$J(\mathbf{w}, b) = \sum_i \log\bigl(1 + e^{-z_i(\mathbf{w}^T \mathbf{x}_i + b)}\bigr),$$

a **discriminative** criterion with **no closed form**, solved numerically. Both live in the same hypothesis class (linear log-odds, which a common-covariance Gaussian implies); they only disagree on which hyperplane to select.

Assumptions, Thresholds and Extensions

Assumptions. LDA assumes **Gaussian classes with a common covariance**; LR assumes only a **linear log-odds** with no model of the feature density, making it more robust when the Gaussian assumption fails. **Threshold/priors.** LDA classifies against the **midpoint** threshold $t = \frac{m_1+m_2}{2}$ (optimal under equal priors); LR's score reflects the **empirical training prior** and can be recalibrated to a log-likelihood ratio for arbitrary applications. **Non-linearity.** LR extends to non-linear boundaries via a feature expansion $\phi(\mathbf{x})$; LDA, being geometric/eigen-based, does not extend naturally. **Dual role / multiclass.** LDA is also a **dimensionality-reduction** technique — at most $K-1$ discriminant directions, then a downstream classifier — whereas LR is purely a classifier and handles multiclass directly via **softmax**.

Aspect	**Logistic Regression**	**LDA (binary)**
Objective	Min cross-entropy / empirical risk (discriminative)	Max Fisher ratio (geometric)
Training	Numerical, no closed form	Closed form (eigenproblem / $S_W^{-1}\Delta\boldsymbol{\mu}$)
Assumptions	Linear log-odds; no model of f_X	Gaussian classes, common covariance
Decision rule	$s \gtrless 0$ (recalibratable to LLR)	$\mathbf{w}^T\mathbf{x} \lessgtr t = \frac{m_1+m_2}{2}$ (midpoint)
Dual use	Classifier only	Also dimensionality reduction ($\leq K-1$ dir.)
Non-linearity	Yes, via $\phi(\mathbf{x})$	No (geometric)
Multiclass	Softmax (direct)	Dim. reduction $\leq K-1$ + downstream classifier

9 Support Vector Machines

The **Support Vector Machine** (SVM) is a **discriminative**, **non-probabilistic** binary classifier that gives a **geometric interpretation of the regularization term** of Logistic Regression. Among the infinitely many hyperplanes that separate two linearly separable classes, it selects the one with the **largest margin**, and through the **kernel trick** it achieves non-linear separation *without* an explicit feature expansion. Its output **cannot be interpreted as a class posterior**.

Maximum-Margin Hyperplane

Let $f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$ be the separating surface and encode labels as $z_i \in \{+1, -1\}$ ($z_i = +1$ for $\mathcal{H}_T$, $z_i = -1$ for $\mathcal{H}_F$). The distance of a point from the hyperplane is $d(\mathbf{x}_i) = \frac{|f(\mathbf{x}_i)|}{\|\mathbf{w}\|}$, and the **margin** is the distance of the closest point. The maximum-margin hyperplane maximizes the minimum distance:

$$\mathbf{w}^*, b^* = \arg\max_{\mathbf{w},b} \min_i \frac{|z_i(\mathbf{w}^T\mathbf{x}_i + b)|}{\|\mathbf{w}\|}.$$

Since the classes are separable, an optimal solution has $z_i(\mathbf{w}^T\mathbf{x}_i + b) > 0\ \forall i$, so the absolute value can be dropped: $\arg\max_{\mathbf{w},b} \frac{1}{\|\mathbf{w}\|} \min_i [z_i(\mathbf{w}^T\mathbf{x}_i + b)]$.

Motivation. Logistic Regression picks the hyperplane maximizing the posteriors, but given any separating $\mathbf{w}$ one can lower the loss by keeping the orientation and *growing* $\|\mathbf{w}\|$ (posteriors $\to 0/1$). The margin gives a well-defined criterion instead.

Canonical Form and Primal Problem

The objective is **invariant under rescaling** $(\mathbf{w}, b) \to (\alpha\mathbf{w}, \alpha b)$, so each solution is an equivalence class. We fix the representative by requiring $\min_i z_i(\mathbf{w}^T\mathbf{x}_i + b) = 1$, hence $z_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1\ \forall i$. The problem becomes the **primal SVM**:

$$\arg\min_{\mathbf{w},b} \tfrac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad z_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1,\ i = 1 \ldots n.$$

At the optimum the constraint is **active** ($\min_i = 1$): a solution with $\min_i z_i(\mathbf{w}^T\mathbf{x}_i + b) = \psi > 1$ could be rescaled by $1/\psi$, satisfying all constraints with smal-

ler norm. This is a **convex quadratic programming** problem (convex objective, convex feasible set), so every local minimum is global.

Lagrangian, Dual Problem and Support Vectors

Introducing multipliers $\alpha_i \geq 0$ for each constraint,

$$L(\mathbf{w}, b, \boldsymbol{\alpha}) = \tfrac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{n} \alpha_i \big[z_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \big].$$

Setting $\nabla_{\mathbf{w}} L = 0$ and $\partial L / \partial b = 0$ gives $\mathbf{w} = \sum_i \alpha_i z_i \mathbf{x}_i$ and $\sum_i \alpha_i z_i = 0$. Substituting yields the **dual problem**:

$$\max_{\boldsymbol{\alpha}} \ \boldsymbol{\alpha}^T \mathbf{1} - \tfrac{1}{2} \boldsymbol{\alpha}^T \mathbf{H} \boldsymbol{\alpha}, \qquad H_{ij} = z_i z_j \, \mathbf{x}_i^T \mathbf{x}_j,$$

$$\text{s.t.} \quad \alpha_i \geq 0, \qquad \sum_{i=1}^{n} \alpha_i z_i = 0.$$

Crucially, $\mathbf{H}$ depends on the data **only through the dot products** $\mathbf{x}_i^T \mathbf{x}_j$: this is what enables kernels. **Weak/strong duality**: $L_D(\boldsymbol{\alpha}) \leq L_P(\mathbf{w}, b)$ for any feasible pair, and the **duality gap** $L_P - L_D \geq 0$ vanishes at the optimum ($L_D(\boldsymbol{\alpha}^*) = L_P(\mathbf{w}^*, b^*)$).

A solution is optimal iff it satisfies the **KKT conditions**: stationarity in $\mathbf{w}$ and b, primal feasibility $z_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \geq 0$, dual feasibility $\alpha_i \geq 0$, and **complementary slackness** $\alpha_i [z_i(\mathbf{w}^T \mathbf{x}_i + b) - 1] = 0$. The last one implies that points strictly outside the margin ($z_i(\mathbf{w}^T \mathbf{x}_i + b) > 1$) have $\alpha_i = 0$, while $\alpha_i \neq 0$ means the point lies **on the margin**: these are the **support vectors**. Non-support vectors do not affect the separating surface; b is recovered from the KKT conditions once $\boldsymbol{\alpha}$ is known.

Soft Margin (Non-Separable Classes)

When classes are not separable, some constraints are always violated. We introduce **slack variables** $\xi_i \geq 0$ relaxing the constraints to $z_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$. Choosing $\sigma = 1$, $F(u) = u$ in the general penalty $\frac{1}{2}\|\mathbf{w}\|^2 + C F(\sum_i \xi_i^\sigma)$ gives the **soft-margin** objective:

$$\min_{\mathbf{w}, b, \boldsymbol{\xi}} \tfrac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \xi_i \quad \text{s.t.} \quad z_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

The ξ_i are a **penalty** (not a count): points inside the margin have $\xi_i > 0$, mis-classified points $\xi_i > 1$, and $\sum_i \xi_i$ is an **upper bound on the number of errors**. The constant C trades off margin width against training errors; the result is a **soft-margin hyperplane**. The dual is unchanged except for a **box constraint** $0 \le \alpha_i \le C$ (besides $\sum_i \alpha_i z_i = 0$).

Hinge Loss and Comparison with Logistic Regression

On correctly classified points $\xi_i = 0$, otherwise $\xi_i = 1 - z_i(\mathbf{w}^T\mathbf{x}_i + b)$, so the constraints are absorbed into the unconstrained primal

$$\min_{\mathbf{w},b} \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \max\big[0,\, 1 - z_i(\mathbf{w}^T\mathbf{x}_i + b)\big],$$

where $f(s) = \max(0, 1-s)$ is the **hinge loss**. In regularized-risk form SVM and LR share the same structure and differ *only* in the per-sample loss:

$$\text{SVM: } \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_i \max\big[0, 1 - z_i s_i\big], \qquad \text{LR: } \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_i \log\big(1 + e^{-z_i s_i}\big),$$

with $s_i = \mathbf{w}^T\mathbf{x}_i + b$. The hinge loss is **exactly zero** for points correctly classified beyond the margin (giving sparsity / support vectors), while the log-loss is always positive.

Inference and the Kernel Trick

The score can be computed in two ways:

$$\text{primal: } s(\mathbf{x}_t) = \mathbf{w}^T\mathbf{x}_t + b \;\; (O(D)), \qquad \text{dual: } s(\mathbf{x}_t) = \sum_{i:\alpha_i>0} \alpha_i z_i\, \mathbf{x}_i^T\mathbf{x}_t + b \;\; (O(\#SV)).$$

The **decision rule** is the sign $s(\mathbf{x}_t) \gtrless 0$. Both forms depend on the data **only through dot products**. A **non-linear** mapping $\mathbf{\Phi} : \mathbb{R}^D \to \mathcal{H}$ enters the dual and the score only as $\mathbf{\Phi}(\mathbf{x}_i)^T\mathbf{\Phi}(\mathbf{x}_j)$, so we replace it by a **kernel** $k(\mathbf{x}_1, \mathbf{x}_2) = \mathbf{\Phi}(\mathbf{x}_1)^T\mathbf{\Phi}(\mathbf{x}_2)$ without ever building $\mathbf{\Phi}$:

$$H_{ij} = z_i z_j\, k(\mathbf{x}_i, \mathbf{x}_j), \qquad s(\mathbf{x}_t) = \sum_{i:\alpha_i>0} \alpha_i z_i\, k(\mathbf{x}_i, \mathbf{x}_t) + b.$$

A linear separation in the expanded space is a **non-linear** separation in the original space, and the dual cost depends only on N, not on $\dim \mathcal{H}$. Common kernels: the **polynomial** kernel $k(\mathbf{x}_1, \mathbf{x}_2) = (\mathbf{x}_1^T \mathbf{x}_2 + 1)^d$ (e.g. $d = 2$ from $\mathbf{\Phi}(\mathbf{x}) = [\text{vec}(\mathbf{x}\mathbf{x}^T); \sqrt{2}\mathbf{x}; 1]$) and the **RBF / Gaussian** kernel $k(\mathbf{x}_1, \mathbf{x}_2) = e^{-\gamma \|\mathbf{x}_1 - \mathbf{x}_2\|^2}$, associated to an **infinite-dimensional** mapping (γ small $\Rightarrow$ wide kernel, a support vector influences far points; γ large $\Rightarrow$ narrow, local influence). **Mercer's condition** is a *sufficient* condition for k to be a dot product in some space: k symmetric and $\int k(\mathbf{u}, \mathbf{v}) g(\mathbf{u}) g(\mathbf{v})\, d\mathbf{u}\, d\mathbf{v} > 0$ for all square-integrable g. It does not say how to build kernels; one uses known kernels or composition rules (e.g. a sum of kernels is a kernel).

Practical Considerations and Limitations

The kernel, its hyper-parameters and C are selected by **cross-validation**; sometimes a linear classifier suffices (high-dimensional, almost separable features). SVM training is **not invariant under affine transformations**, so centering and whitening usually help. SVM scores have **no probabilistic interpretation**: a post-processing/**calibration** step is needed to obtain posteriors. For **class imbalance** (training prior $\neq$ target prior) one uses class-dependent costs C_i,

$$\arg\min_{\mathbf{w},b} \tfrac{1}{2}\|\mathbf{w}\|^2 + \sum_{i=1}^{n} C_i \big[1 - z_i(\mathbf{w}^T \mathbf{x}_i + b)\big]_+, \qquad C_T = C \tfrac{\pi_T}{\pi_T^{emp}}, \quad C_F = C \tfrac{\pi_F}{\pi_F^{emp}},$$

simulating the application priors at training time (still without a probabilistic score). Finally, SVM is natively binary and **hard to extend to multiclass**. Kernels are not exclusive to SVMs: any method depending only on dot products (e.g. Logistic Regression) can be kernelized.

[THEORETICAL INTEGRATION]
Why support vectors are only the points on the margin. The complementary slackness condition $\alpha_i[z_i(\mathbf{w}^T \mathbf{x}_i + b) - 1] = 0$ requires that, for each point, at least one of α_i and the *gap* $z_i(\mathbf{w}^T \mathbf{x}_i + b) - 1$ is zero. Therefore a point strictly outside the margin (gap > 0) must have $\alpha_i = 0$ and, since $\mathbf{w} = \sum_i \alpha_i z_i \mathbf{x}_i$, *does not contribute* to the solution. Only points with zero gap (on the margin) can have $\alpha_i > 0$: these are the support vectors, the only ones that define the hyperplane.

Alternative Version (ordered according to the exam outline)

Same theory, reorganized following the order of points required by the exam question: (1) classification rule and score interpretation, (2) margin, (3) primal (QP and hinge loss) and dual, with explicit relation between the two solutions, (4) non-linear SVMs.

1. Classification Rule and Score Interpretation

SVM is a **discriminative, non-probabilistic** classifier: it computes a score $s(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$ and assigns the class based on its sign,

$$\text{decision: } z = +1 \text{ if } s(\mathbf{x}) > 0, \qquad z = -1 \text{ if } s(\mathbf{x}) < 0.$$

The score is the **signed distance from the separating hyperplane**, scaled by $\|\mathbf{w}\|$: it is *not* a log-likelihood ratio nor a posterior, so it **cannot be interpreted probabilistically** without a subsequent calibration step. Its absolute value does indicate how far a point is from the boundary (and thus how confidently it is classified), making it useful for applying thresholds other than 0 depending on application costs.

2. Concept of Margin

Given $f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$ and $z_i \in \{+1, -1\}$, the distance of a point from the hyperplane is $d(\mathbf{x}_i) = \dfrac{|f(\mathbf{x}_i)|}{\|\mathbf{w}\|}$. The **margin** is the minimum distance between the hyperplane and the training points:

$$\mathbf{w}^*, b^* = \arg\max_{\mathbf{w},b} \min_i \frac{|z_i(\mathbf{w}^T\mathbf{x}_i + b)|}{\|\mathbf{w}\|} = \arg\max_{\mathbf{w},b} \frac{1}{\|\mathbf{w}\|} \min_i \left[z_i(\mathbf{w}^T\mathbf{x}_i + b)\right]$$

(the second form holds because, on a separating solution, $z_i(\mathbf{w}^T\mathbf{x}_i + b) > 0 \; \forall i$). **Motivation**: Logistic Regression maximizes the posteriors, but given any orientation of $\mathbf{w}$ one can always reduce the loss by increasing $\|\mathbf{w}\|$ (posteriors tend to 0/1), with no natural stopping criterion. The margin provides a well-defined geometric criterion, independent of the scale of $\mathbf{w}$.

3. Primal and Dual, and Relation Between the Solutions

(a) Primal – constrained QP form. The objective is invariant under rescaling $(\mathbf{w}, b) \to (\alpha\mathbf{w}, \alpha b)$: we fix the representative by requiring $\min_i z_i(\mathbf{w}^T\mathbf{x}_i + b) = 1$. For the **separable** case (hard margin):

$$\arg\min_{\mathbf{w},b} \tfrac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad z_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1,\ i = 1 \dots n.$$

This is a **convex quadratic programming** problem (convex objective, convex feasible set): every local minimum is global.

For the **non-separable** case, **slack variables** $\xi_i \geq 0$ are introduced, relaxing the constraints to $z_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i$:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \xi_i \quad \text{s.t.} \quad z_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

The ξ_i are a **penalty** (not a count): points inside the margin have $\xi_i > 0$, misclassified points have $\xi_i > 1$, and $\sum_i \xi_i$ is an **upper bound on the number of errors**. C trades off margin width against training errors.

(b) Primal – hinge loss form. On correctly classified points $\xi_i = 0$, otherwise $\xi_i = 1 - z_i(\mathbf{w}^T\mathbf{x}_i + b)$: the constraints are absorbed into an **unconstrained** problem

$$\min_{\mathbf{w},b} \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \max\big[0,\ 1 - z_i(\mathbf{w}^T\mathbf{x}_i + b)\big],$$

where $f(s) = \max(0, 1-s)$ is the **hinge loss**. In *regularized risk* form, SVM and LR share the same structure and differ only in the per-sample loss:

$$\text{SVM: } \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_i \max\big[0, 1 - z_i s_i\big], \qquad \text{LR: } \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_i \log\big(1 + e^{-z_i s_i}\big), \quad s_i = \mathbf{w}^T\mathbf{x}_i + b.$$

The hinge loss is **exactly zero** for points correctly classified beyond the margin (giving **sparsity** of support vectors), while the log-loss is always positive.

(c) Dual. Introducing Lagrange multipliers $\alpha_i \geq 0$,

$$L(\mathbf{w}, b, \boldsymbol{\alpha}) = \tfrac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{n} \alpha_i \big[z_i(\mathbf{w}^T\mathbf{x}_i + b) - 1\big].$$

From $\nabla_{\mathbf{w}} L = 0$ and $\partial L/\partial b = 0$ one gets $\mathbf{w} = \sum_i \alpha_i z_i \mathbf{x}_i$ and $\sum_i \alpha_i z_i = 0$. Substituting, the **dual problem** is

$$\max_{\boldsymbol{\alpha}} \ \boldsymbol{\alpha}^T \mathbf{1} - \tfrac{1}{2}\boldsymbol{\alpha}^T \mathbf{H} \boldsymbol{\alpha}, \qquad H_{ij} = z_i z_j \, \mathbf{x}_i^T \mathbf{x}_j, \qquad \text{s.t.} \quad \sum_{i=1}^{n} \alpha_i z_i = 0, \quad \begin{cases} \alpha_i \geq 0 & \text{(hard margin)} \\ 0 \leq \alpha_i \leq C & \text{(soft margin)} \end{cases}$$

$\mathbf{H}$ depends on the data **only through the dot products** $\mathbf{x}_i^T \mathbf{x}_j$: this is what enables the **kernel trick** (point 4).

(d) Relation between primal and dual solutions. By **weak/strong duality**, $L_D(\boldsymbol{\alpha}) \leq L_P(\mathbf{w}, b)$ for any feasible pair, and the **duality gap** $L_P - L_D \geq 0$ vanishes at the optimum: $L_D(\boldsymbol{\alpha}^*) = L_P(\mathbf{w}^*, b^*)$. The primal solution is recovered *exactly* from the dual via the stationarity conditions:

$$\mathbf{w}^* = \sum_{i=1}^{n} \alpha_i^* z_i \mathbf{x}_i, \qquad b^* = \text{recovered from KKT (from any support vector with } 0 < \alpha_i^* < C).$$

The **KKT conditions** (stationarity, primal feasibility $z_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \geq 0$, dual feasibility $\alpha_i \geq 0$, and **complementary slackness** $\alpha_i \big[z_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \big] = 0$) characterize the optimum: for each point, either $\alpha_i = 0$ or the constraint is active ($z_i(\mathbf{w}^T \mathbf{x}_i + b) = 1$). Therefore only the points **on the margin** have $\alpha_i \neq 0$ – the **support vectors** – and they are the only ones contributing to $\mathbf{w}^*$; points strictly outside the margin have $\alpha_i = 0$ and **do not affect** the solution.

4. Non-Linear SVMs (Kernel Trick)

Both at training time (dual) and at inference, the data appear only through dot products:

$$\text{primal: } s(\mathbf{x}_t) = \mathbf{w}^T \mathbf{x}_t + b \ \ (O(D)), \qquad \text{dual: } s(\mathbf{x}_t) = \sum_{i:\alpha_i > 0} \alpha_i z_i \, \mathbf{x}_i^T \mathbf{x}_t + b \ \ (O(\#SV)).$$

A non-linear map $\boldsymbol{\Phi} : \mathbb{R}^D \to \mathcal{H}$ enters the dual and the score only as $\boldsymbol{\Phi}(\mathbf{x}_i)^T \boldsymbol{\Phi}(\mathbf{x}_j)$: it is replaced by a **kernel** $k(\mathbf{x}_1, \mathbf{x}_2) = \boldsymbol{\Phi}(\mathbf{x}_1)^T \boldsymbol{\Phi}(\mathbf{x}_2)$ without ever building $\boldsymbol{\Phi}$ explicitly:

$$H_{ij} = z_i z_j \, k(\mathbf{x}_i, \mathbf{x}_j), \qquad s(\mathbf{x}_t) = \sum_{i:\alpha_i > 0} \alpha_i z_i \, k(\mathbf{x}_i, \mathbf{x}_t) + b.$$

A linear separation in the expanded space corresponds to a **non-linear** separation in the original space, and the dual cost depends only on N,

not on $\dim \mathcal{H}$. Common kernels: **polynomial** $k(\mathbf{x}_1, \mathbf{x}_2) = (\mathbf{x}_1^T \mathbf{x}_2 + 1)^d$ and **RBF/Gaussian** $k(\mathbf{x}_1, \mathbf{x}_2) = e^{-\gamma \|\mathbf{x}_1 - \mathbf{x}_2\|^2}$, associated with an infinite-dimensional mapping (γ small $\Rightarrow$ wide influence, γ large $\Rightarrow$ local influence). **Mercer's condition** (k symmetric and $\int k(\mathbf{u}, \mathbf{v}) g(\mathbf{u}) g(\mathbf{v})\, d\mathbf{u}\, d\mathbf{v} > 0 \ \forall g$) is a *sufficient* condition for k to be a dot product in some space.

Third Version (lecture notes)

Classification Rule

The SVM classifier is a discriminative, non-probabilistic approach for binary classification. Given a trained model with parameters $(\mathbf{w}, b)$, the classification rule for a test sample $\mathbf{x}_t$ consists of computing a linear decision function and comparing it with a threshold of 0:

$$s(\mathbf{x}_t) = \mathbf{w}^T\mathbf{x}_t + b, \qquad \text{decision: } \operatorname{sign}(s(\mathbf{x}_t)),$$

where class labels are encoded as $z \in \{-1, +1\}$ ($z = +1$ for $\mathcal{H}_T$, $z = -1$ for $\mathcal{H}_F$). The continuous value $s = \mathbf{w}^T\mathbf{x} + b$ is the geometric score: its sign dictates the predicted class and its magnitude represents the distance from the separating surface. The SVM score has no probabilistic interpretation.

Concept of Margin

The distance between a training point $\mathbf{x}_i$ and the separating hyperplane $\mathbf{w}^T\mathbf{x} + b = 0$ is

$$d(\mathbf{x}_i) = \frac{|\mathbf{w}^T\mathbf{x}_i + b|}{\|\mathbf{w}\|}.$$

The margin is the distance between the closest data point and the hyperplane. SVM selects the unique solution that maximizes this minimum distance:

$$\mathbf{w}^*, b^* = \arg\max_{\mathbf{w},b} \min_i \frac{z_i(\mathbf{w}^T\mathbf{x}_i + b)}{\|\mathbf{w}\|}.$$

Since the classes are separable, $z_i(\mathbf{w}^T\mathbf{x}_i + b) > 0$ on an optimal solution, so this reduces to $\arg\max_{\mathbf{w},b} \frac{1}{\|\mathbf{w}\|} \min_i[z_i(\mathbf{w}^T\mathbf{x}_i + b)]$. This criterion defines a well-defined geometric objective.

Primal Formulation

Maximizing the margin $1/\|\mathbf{w}\|$ is mathematically equivalent to minimizing $\frac{1}{2}\|\mathbf{w}\|^2$. For the separable (hard margin) case:

$$\min_{\mathbf{w},b} \tfrac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad z_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1, \ i = 1 \dots n.$$

This is a convex quadratic programming problem where every local minimum is global.

When classes are not separable, slack variables $\xi_i \geq 0$ are introduced to relax the constraints to $z_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i$. This yields the soft margin problem:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i \quad \text{s.t.} \quad z_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0,$$

where C is a hyperparameter trading off margin width against training errors.

Since for any point $\xi_i = \max[0,\, 1 - z_i(\mathbf{w}^T\mathbf{x}_i + b)]$, the constraints are absorbed into an unconstrained primal:

$$\min_{\mathbf{w},b} \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \max\big[0,\, 1 - z_i(\mathbf{w}^T\mathbf{x}_i + b)\big],$$

where $\ell(s) = \max(0, 1 - s)$ is the **hinge loss**.

Dual Formulation

By introducing Lagrange multipliers $\alpha_i \geq 0$ for each constraint, the Lagrangian is:

$$L(\mathbf{w}, b, \boldsymbol{\alpha}) = \tfrac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{n} \alpha_i \big[z_i(\mathbf{w}^T\mathbf{x}_i + b) - 1\big].$$

Setting $\nabla L = 0$ yields $\mathbf{w} = \sum_i \alpha_i z_i \mathbf{x}_i$ and $\sum_i \alpha_i z_i = 0$. Substituting gives the Wolfe dual problem:

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \tfrac{1}{2}\sum_i \sum_j \alpha_i \alpha_j z_i z_j \mathbf{x}_i^T \mathbf{x}_j \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i z_i = 0.$$

The connection between primal and dual solutions is dictated by the KKT conditions, specifically complementary slackness:

$$\alpha_i \big[z_i(\mathbf{w}^T\mathbf{x}_i + b) - 1\big] = 0.$$

For every sample, either $\alpha_i = 0$ or the margin gap is zero. All points outside the margin have $\alpha_i = 0$; since $\mathbf{w} = \sum_i \alpha_i z_i \mathbf{x}_i$, these points do not contribute to the orientation of the hyperplane. Only points exactly on the margin or violating it have $\alpha_i > 0$: these are the **support vectors**, and they alone determine the separating surface. The solution is sparse in the training set.

Non-linear SVM and Kernel Trick

From the dual problem, the model can be naturally kernelized through its data dot products $\mathbf{x}_i^T \mathbf{x}_j$. The kernel trick maps data points into a high-dimensional feature space $\mathbf{\Phi}(\mathbf{x})$ by substituting the inner product with a non-linear kernel function:

$$k(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{\Phi}(\mathbf{x}_i)^T \mathbf{\Phi}(\mathbf{x}_j), \qquad \mathbf{\Phi} : \mathbb{R}^D \to \mathcal{H},$$

with $H_{ij} = z_i z_j \, k(\mathbf{x}_i, \mathbf{x}_j)$. The non-linear classification score depends only on the sparse subset of support vectors:

$$s(\mathbf{x}_t) = \sum_{i:\alpha_i > 0} \alpha_i z_i \, k(\mathbf{x}_i, \mathbf{x}_t) + b.$$

A linear separation in the expanded space corresponds to a non-linear separation in the original space, and the dual cost depends only on N, not on $\dim \mathcal{H}$. Common kernels: polynomial $k(\mathbf{x}_1, \mathbf{x}_2) = (\mathbf{x}_1^T \mathbf{x}_2 + 1)^d$ and RBF $k(\mathbf{x}_1, \mathbf{x}_2) = e^{-\gamma \|\mathbf{x}_1 - \mathbf{x}_2\|^2}$, associated with an infinite-dimensional mapping. Mercer's condition is sufficient for a function k to be a valid dot product in some inner product space.

Comparison with Logistic Regression

Both LR and SVM can be interpreted as regularized risk minimization. Logistic Regression uses the smooth, continuous logistic loss:

$$R(\mathbf{w}, b) = \tfrac{\lambda}{2} \|\mathbf{w}\|^2 + \tfrac{1}{n} \sum_{i=1}^{n} \log\bigl(1 + e^{-z_i s_i}\bigr).$$

SVM uses the piecewise linear hinge loss:

$$R(\mathbf{w}, b) = \tfrac{\lambda}{2} \|\mathbf{w}\|^2 + \tfrac{1}{n} \sum_{i} \max(0,\, 1 - z_i s_i).$$

In LR, λ is required to guarantee parameter convergence on separable data. In SVM, λ minimizes $\frac{1}{2}\|\mathbf{w}\|^2$, which corresponds to maximizing the geometric margin.

The logistic loss is always positive, so in LR every sample contributes to the solution (not sparse). The hinge loss is exactly zero for each sample correctly classified beyond the margin, so only points inside the margin determine the solution: SVM is sparse in the training set.

LR returns a calibrated posterior $P(\mathcal{C}|\mathbf{x})$, with the score being a log-posterior-ratio. SVM has no probabilistic meaning and needs an explicit calibration step.

10 Logistic Regression vs SVM

Logistic Regression (LR) and the **Support Vector Machine** (SVM) are the two prototypical **discriminative, linear** classifiers cast as **regularized risk minimization**. They learn the same kind of score $s = \mathbf{w}^T\mathbf{x} + b$ and classify by its sign; they differ essentially in *one ingredient* — the per-sample loss — and in whether the output is probabilistic.

The Shared Risk-Minimization Form

Both minimize an objective of the form $\frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_i \ell(z_i s_i)$, with $z_i \in \{\pm 1\}$ and $s_i = \mathbf{w}^T\mathbf{x}_i + b$, differing **only in the loss**:

LR: $\ell(z_i s_i) = \log\big(1 + e^{-z_i s_i}\big)$, SVM: $\ell(z_i s_i) = \max\big(0,\, 1 - z_i s_i\big)$ (hinge).

Neither models the feature density $f_X(\mathbf{x})$. The **regularization term** $\frac{\lambda}{2}\|\mathbf{w}\|^2$ plays the same role in both, but with complementary readings: in LR it is needed because on **separable** data the log-loss has no minimum (driving $\|\mathbf{w}\| \to \infty$); in SVM the *same* term **is** the objective, and minimizing $\frac{1}{2}\|\mathbf{w}\|^2$ subject to $z_i s_i \geq 1$ is exactly **maximizing the margin**. SVM is thus the **geometric interpretation** of LR's regularization.

Loss Shape, Support Vectors and Probabilities

The hinge loss is **exactly zero** for points correctly classified beyond the margin: only the points on/inside the margin (the **support vectors**, $\alpha_i \neq 0$) determine the solution, so SVM is **sparse** in the training set. The log-loss is **always positive**, so in LR **every** sample contributes. The other central difference is the **output**: LR returns a calibrated **posterior** $\sigma(s)$ with s a log-posterior-ratio, whereas the SVM score has **no probabilistic meaning** and needs an explicit **calibration** step before it can be used as an LLR.

Optimization, Non-linearity, Priors and Multiclass

LR is solved by **direct numerical minimization** of the primal (loss + gradient); SVM is a **convex QP**, usually solved through its **dual** (Lagrangian + KKT conditions), which both exposes the support vectors and enables kernels. **Non-linearity** is reached differently: LR uses an **explicit** feature expansion $\phi(\mathbf{x})$, SVM uses the **kernel trick** $k(\mathbf{x}_i, \mathbf{x}_j)$ (implicit, possibly infinite-dimensional) — though kernels are not exclusive to SVM and LR can be kernelized too. On **priors**, neither score natively matches the application prior: LR is **recalibrated** ($s_{llr} = s - \log \frac{n_T}{n_F}$) or trained prior-weighted, while SVM rebalances through class-dependent costs C_T, C_F. Finally, LR extends to **multiclass** naturally via **softmax**, whereas SVM is natively binary and hard to extend.

Aspect	**Logistic Regression**	**Support Vector Machine**
Objective	$\frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum \log(1 + e^{-z_i s_i})$	$\frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum \max(0, 1 - z_i s_i)$
Loss (per sample)	log / softplus (always > 0)	hinge (zero beyond the margin)
Training	ML / cross-entropy, numerical solver	convex QP, dual + KKT
Output / score	posterior $\sigma(s)$; s is a log-odds	sign of s; **no** probabilistic meaning
Decision rule	$s \gtrless 0$; Bayes threshold after recalibration	$s \gtrless 0$; no native Bayes threshold
Non-linearity	explicit feature expansion $\phi(\mathbf{x})$	implicit kernel k (even ∞-dim)
Sparsity	no (all samples contribute)	yes (only support vectors)
Multiclass	natural (softmax)	hard (binary by nature)

11 Generative Gaussian Models vs SVM

The **Generative Gaussian Models** (MVG) and the **Support Vector Machine** (SVM) sit at opposite ends of the classifier spectrum: a **generative, probabilistic** model versus a **discriminative, geometric, non-probabilistic** one. They can produce the *same* decision boundaries, but obtain and interpret them in completely different ways.

Same Boundaries, Opposite Constructions

The **Tied** MVG has a **linear** log-posterior-ratio $s = \mathbf{w}^T\mathbf{x} + b$ with $\mathbf{w} \propto \mathbf{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)$, the same *form* as the linear SVM; the **full** MVG (QDA) gives

a **quadratic** boundary, matched by an SVM with a degree-2 **polynomial kernel**. So the two can span the same family of surfaces. The difference is *how*: MVG models the class-conditional densities $f(\mathbf{x} \mid c)P(c)$ and gets the boundary through **Bayes**, estimating $\mathbf{w}$ from **ML Gaussian statistics**; SVM never models a density and obtains the boundary by **maximizing the margin**.

Probabilities, Training and What the Data Are Used For

MVG produces a **log-likelihood ratio** / posterior, **Bayes-ready**: the decision rule is $\text{llr} \gtrless -\log \frac{P(c_1)}{P(c_0)}$ and priors/costs can be changed at deployment by moving the threshold. The SVM score has **no probabilistic interpretation**: it needs **calibration** and has no native Bayes threshold; imbalance is handled through class-dependent costs C_T, C_F. **Training** also diverges: MVG is **closed-form** (per-class means and covariances), SVM is a **convex QP** (dual + KKT, no closed form). Finally, MVG uses **aggregate statistics of all points**, whereas the SVM solution depends **only on the support vectors** and on **dot products**, hence is kernelizable.

Assumptions, Non-linearity and Multiclass

MVG makes a **strong Gaussian assumption** on the class-conditionals (data-efficient when it holds, requiring $\mathbf{\Sigma}$ invertible, $N > D$, often PCA first); SVM makes **no density assumption** and is more robust to non-Gaussian features. For **non-linear** boundaries MVG uses a full covariance (quadratic, with $\frac{D(D+1)}{2}$ parameters per class), while SVM uses the **kernel trick** (even infinite-dimensional, without building $\mathbf{\Phi}$). MVG is **natively multiclass** (the $\arg\max$ of the log-posterior over K classes), whereas SVM is binary by nature and **hard to extend**.

Aspect	Generative Gaussian (MVG / Tied)	Support Vector Machine
Type	Generative: $f(\mathbf{x}\,\vert\,c)P(c)$, then Bayes	Discriminative, non-probabilistic
Objective	ML of Gaussian class-conditionals	max margin / hinge $+\frac{1}{2}\|\mathbf{w}\|^2$
Assumptions	Gaussian class-conditionals; $\boldsymbol{\Sigma}$ invertible	none on the feature density
Training	closed form, per class	convex QP (dual, KKT)
Score	LLR $\log \frac{f(\mathbf{x}\vert c_1)}{f(\mathbf{x}\vert c_0)}$; posterior via Bayes	$\sum_{\text{SV}} \alpha_i z_i k(\mathbf{x}_i, \mathbf{x}) + b$; no posterior
Decision rule	llr $\gtrless -\log \frac{P(c_1)}{P(c_0)}$; quadratic (linear if tied)	$s \gtrless 0$; linear or kernel-nonlinear
Priors	separated $\Rightarrow$ threshold/costs native	no native prior; costs C_T, C_F
Multiclass	native ($\arg\max$ log-posterior)	hard (binary by nature)

12 Gaussian Mixture Models

Motivation. A **Gaussian Mixture Model (GMM)** is a density model obtained as a **weighted combination of Gaussians**. By combining several simple Gaussian components it can approximate **any sufficiently regular distribution** to a desired degree, provided enough data. GMMs are used for **density estimation**, **clustering** (a generalization of K-means), and as **generative classifiers** (one GMM per class). The starting observation is that, in a Gaussian classifier, the marginal density $f_X(x) = \sum_c \pi_c \mathcal{N}(x; \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$ is *already* a GMM.

Mathematical formulation. A K-component GMM is:

$$f_X(x) = \sum_{c=1}^{K} w_c \mathcal{N}(x; \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c), \qquad w_c \geq 0, \quad \sum_{c=1}^{K} w_c = 1$$

The constraint $\sum_c w_c = 1$ follows from requiring $\int f_X(x)\,dx = 1$. The parameters are $\theta = (\mathcal{M}, \mathcal{S}, \mathbf{w})$ with $\mathcal{M} = \{\boldsymbol{\mu}_c\}$, $\mathcal{S} = \{\boldsymbol{\Sigma}_c\}$, $\mathbf{w} = \{w_c\}$.

Latent-variable interpretation. A GMM is the **marginal** of a joint distribution over a sample X_i and a hidden **cluster** variable C_i:

$$f_{X_i}(x_i) = \sum_{c=1}^{K} f_{X_i|C_i}(x_i|c)\, P(C_i = c) = \sum_{c=1}^{K} w_c \mathcal{N}(x_i; \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$$

with categorical prior $P(C_i = c) = w_c$ and conditional $f_{X_i|C_i}(x_i|c) = \mathcal{N}(x_i; \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$. The cluster membership is a **latent (hidden) variable**: it is not observed but governs the data-generation process.

Likelihood and ill-posed ML. For an i.i.d. dataset $\mathcal{D} = \{x_1, \ldots, x_N\}$ (treated as **unlabeled**), the log-likelihood is:

$$\ell(\theta) = \sum_{i=1}^{N} \log \sum_{c=1}^{K} w_c \, \mathcal{N}(x_i; \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$$

The **log of a sum** has no closed-form maximizer. Moreover, ML for GMMs is **ill-posed**: with at least two components the likelihood is **unbounded above** (a component can collapse onto a single point, $\boldsymbol{\Sigma}_c \to 0$), so degenerate solutions exist. Combined with heuristics, ML still yields good density estimates.

Responsibilities and hard assignment. Given θ, the **responsibility** (cluster posterior) of component c for sample x_i is:

$$\gamma_{c,i} = P(C_i = c | X_i = x_i) = \frac{w_c \, \mathcal{N}(x_i; \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)}{\sum_{c'} w_{c'} \, \mathcal{N}(x_i; \boldsymbol{\mu}_{c'}, \boldsymbol{\Sigma}_{c'})}$$

A first, crude approach assigns each point to $\hat{c}_i = \arg\max_c \gamma_{c,i}$ (**hard clustering**) and re-estimates parameters by ML as if labels were known. This ignores uncertainty when clusters overlap and does **not** maximize the observed-data likelihood.

K-means as a special case. If we fix $\boldsymbol{\Sigma}_c = I$ and $w_c = 1/K$, hard assignment reduces to:

$$\hat{c}_i = \arg\max_c P(C_i = c | x_i) = \arg\min_c \|x_i - \boldsymbol{\mu}_c\|^2$$

i.e. assign each point to the nearest centroid and re-estimate centroids — the **K-means** algorithm. K-means is therefore a special case of (hard-assignment, isotropic) GMM training.

Training via EM. Soft assignments are handled by the **Expectation-Maximization (EM)** algorithm, the general method for ML estimation when the likelihood is a marginal $f_X(x) = \int f_{X,H}(x, h)\, dh$ over a latent variable H. Introducing a distribution $Q(h)$, the log-likelihood decomposes as:

$$\log f_X(x; \theta) = \mathcal{L}_h(Q, \theta) + D_{KL}\big(Q \,\|\, f_{H|X}(h|x; \theta)\big), \qquad D_{KL} \geq 0$$

so $\mathcal{L}_h$ is a **lower bound (ELBO)** on the log-likelihood. EM iterates:

- **E-step:** maximize the bound w.r.t. Q by setting $Q^t(h) = f_{H|X}(h|x;\theta^t)$, which makes $D_{KL} = 0$ (the bound becomes tight). It computes the **auxiliary function** $\mathcal{Q}(\theta,\theta^t) = \mathbb{E}_{f_{H|X}(h|x;\theta^t)}[\log f_{X,H}(x,h;\theta)]$.
- **M-step:** maximize the bound w.r.t. θ: $\theta^{t+1} = \arg\max_\theta \mathcal{Q}(\theta,\theta^t)$.

Because the E-step makes the bound tight and the M-step never decreases it, the likelihood is **monotonically non-decreasing**: $\log f_X(x;\theta^{t+1}) \geq \log f_X(x;\theta^t)$. EM converges to a **stationary point** (local maximum / saddle) that depends on the initialization.

EM for GMM. The latent variables are the cluster assignments $h = \{C_1,\dots,C_N\}$, with joint $f_{X_i,C_i}(x_i,c) = w_c\,\mathcal{N}(x_i;\boldsymbol{\mu}_c,\boldsymbol{\Sigma}_c)$. The two steps become:

- **E-step:** compute $\gamma_{c,i} = P(C_i = c|x_i;\theta^t)$. The auxiliary function is

$$\mathcal{Q}(\theta,\theta^t) = \sum_{i=1}^{N}\sum_{c=1}^{K}\gamma_{c,i}\big[\log\mathcal{N}(x_i;\boldsymbol{\mu}_c,\boldsymbol{\Sigma}_c) + \log w_c\big]$$

- **M-step:** maximize w.r.t. θ subject to $\sum_c w_c = 1$:

$$\boldsymbol{\mu}_c^* = \frac{\sum_i \gamma_{c,i}x_i}{\sum_i \gamma_{c,i}},\quad \boldsymbol{\Sigma}_c^* = \frac{\sum_i \gamma_{c,i}(x_i-\boldsymbol{\mu}_c^*)(x_i-\boldsymbol{\mu}_c^*)^T}{\sum_i \gamma_{c,i}},\quad w_c^* = \frac{\sum_i \gamma_{c,i}}{\sum_{i,c'}\gamma_{c',i}}$$

These are the **soft** versions of the hard-assignment ML updates (membership weights $\gamma_{c,i}$ replace $0/1$ labels). Defining the **zero, first and second order statistics** $N_c = \sum_i \gamma_{c,i}$, $F_c = \sum_i \gamma_{c,i}x_i$, $S_c = \sum_i \gamma_{c,i}x_ix_i^T$, the updates read $\boldsymbol{\mu}_c^* = F_c/N_c$, $\boldsymbol{\Sigma}_c^* = S_c/N_c - \boldsymbol{\mu}_c^*\boldsymbol{\mu}_c^{*T}$, $w_c^* = N_c/N$.

Inference: classification. For a closed-set task we fit a GMM with K_c components to each class and apply Bayes:

$$f_{X_t|C_t}(x_t|c) = \sum_{k=1}^{K_c} w_{c,k}\,\mathcal{N}(x_t;\boldsymbol{\mu}_{c,k},\boldsymbol{\Sigma}_{c,k}),\qquad P(C_t=c|x_t) \propto P(C_t=c)\,f_{X_t|C_t}(x_t|c)$$

The decision rule is the usual generative one — binary log-likelihood ratio against a threshold from the log-odds, or $\arg\max_c[\log f_{X|C}(x_t|c) + \log P(C =$

$c)]$ for multiclass — but the class-conditional density is now a **mixture**, so the decision boundary can be an **arbitrary non-linear** surface whose flexibility grows with K_c (a single-component GMM recovers the MVG/quadratic boundary).

Open-set classification. GMMs help model the heterogeneous **"none-of-the-others"** class: known classes may stay MVG, while a GMM trained on a large pool of unlabeled samples discovers homogeneous sub-clusters of the rejection class. Because different numbers of parameters are used, the resulting class-conditional likelihoods are often **not calibrated** and may need further score processing.

Variants.

- **Diagonal covariance:** fewer parameters (less overfitting/cost), possibly needing more components. This is **not** the Naive Bayes assumption: Naive Bayes would train a separate GMM for each independent subset of features.
- **Tied GMM:** all components of a single class's GMM share one $\boldsymbol{\Sigma}$. This differs from the **Tied MVG**, where the covariance is shared *across classes*.
- **Initialization** is crucial (EM finds only local optima). **K-means** can initialize the components; the **LBG** algorithm splits each component $\boldsymbol{\mu}_c^{\pm} = \boldsymbol{\mu}_c \pm \boldsymbol{\varepsilon}$ (a good $\boldsymbol{\varepsilon}$ is a displacement along the principal eigenvector of $\boldsymbol{\Sigma}_c$) to grow a G-component GMM into $2G$ components, running EM after each split.

Limitations. The likelihood is **unbounded** with ≥ 2 components, so EM can reach **degenerate models** ($\boldsymbol{\Sigma}_c \to 0$) causing numerical problems — controlled with heuristics or constrained covariances. EM only reaches **local optima**, hence sensitivity to initialization and the use of multiple restarts. The **number of components** cannot be chosen by maximizing the likelihood (which always grows with K); **cross-validation** is used instead. Finally, reliable density estimation requires a sufficient amount of data.

13 Gaussian Mixture Models vs Generative Gaussian Models

The **Generative Gaussian Models** (MVG) and **Gaussian Mixture Models** (GMM) belong to the *same* family — both are **generative, probabilistic** models of the class-conditional density $f_{X|C}(x|c)$ used through **Bayes** — and in fact the MVG is a **special case** of the GMM. They differ in flexibility, in how the parameters are estimated, and in whether the data are labelled.

One is a Special Case of the Other

A single multivariate Gaussian is exactly a **one-component GMM** ($K = 1 \Rightarrow w_1 = 1$, $f_X = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$); a GMM is a **mixture** of such Gaussians, $f_{X|C}(x|c) = \sum_k w_{c,k}\mathcal{N}(x; \boldsymbol{\mu}_{c,k}, \boldsymbol{\Sigma}_{c,k})$. Conversely, the marginal $f_X(x) = \sum_c \pi_c \mathcal{N}(x; \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$ that an MVG *classifier* forms over its classes is itself a GMM. The **inference and decision rule are identical**: binary log-likelihood ratio against a log-odds threshold, multiclass $\arg\max_c[\log f_{X|C}(x|c) + \log P(C = c)]$ — only the *form* of $f_{X|C}$ changes. Because of this, the MVG produces a **linear** (tied) or **quadratic** (full/QDA) boundary, whereas the GMM produces an **arbitrary non-linear** boundary whose flexibility grows with the number of components K_c.

Closed-form ML versus EM

The MVG has **no latent variable**: labels are observed, and the parameters follow in **closed form** (per-class sample mean and covariance), a single well-posed global optimum. The GMM treats **cluster membership as a latent variable** and is trained by the iterative **EM** algorithm — E-step responsibilities $\gamma_{c,i}$, M-step weighted updates — which only reaches a **local optimum** and is sensitive to initialization (K-means / LBG). The two estimators are directly related: the GMM M-step updates $\boldsymbol{\mu}_c^* = \frac{\sum_i \gamma_{c,i} x_i}{\sum_i \gamma_{c,i}}$, $\boldsymbol{\Sigma}_c^*$, w_c^* are the **soft, responsibility-weighted** version of the MVG closed-form ML, and reduce to it when responsibilities are hard 0/1. Crucially, the GMM likelihood is **ill-posed** (unbounded with ≥ 2 components, degenerate $\boldsymbol{\Sigma}_c \to 0$), whereas the MVG likelihood is bounded and well-behaved.

Supervision, Variants and Model Selection

The MVG ML estimate is **supervised** (it needs labels); the core GMM derivation is **unsupervised** ($\mathcal{D}$ treated as unlabelled), so GMMs also serve for **clustering** and **open-set** modelling of a heterogeneous "none-of-the-others" class, not only closed-set classification. Both reuse the **full** / **diagonal** / **tied** covariance vocabulary, but with different meaning: in the MVG *tied* shares $\boldsymbol{\Sigma}$ **across classes** and *diagonal* is Naive Bayes, while in a GMM *tied* shares $\boldsymbol{\Sigma}$ **across the components of one class** and *diagonal* is **not** Naive Bayes. Finally the GMM adds a **model-selection** problem — the number of components — that cannot be settled by likelihood (which always increases with K) and requires **cross-validation**; the MVG has no such choice.

Aspect	**Generative Gaussian (MVG)**	**Gaussian Mixture Model (GMM)**
Objective	ML of one Gaussian per class	ML of a mixture density (latent clusters)
Class-conditional	$\mathcal{N}(x; \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$	$\sum_k w_{c,k}\mathcal{N}(x; \boldsymbol{\mu}_{c,k}, \boldsymbol{\Sigma}_{c,k})$
Assumptions	class data is Gaussian	weighted mix of Gaussians; cluster = latent
Training	closed-form ML, global optimum	EM (E/M steps), local optimum, init-dependent
Likelihood	bounded, well-posed	unbounded / ill-posed, degeneracy risk
Decision rule	LLR / arg max posterior; linear (tied) or quadratic (full)	LLR / arg max posterior; arbitrary non-linear ($\uparrow K_c$)
Supervision	supervised (labels)	unsupervised core; clustering / open-set
Model selection	none ($K = 1$)	choose #components via cross-validation

14 Bayes Decisions and Model Evaluation

Motivation. Maximum-a-posteriori class assignment ignores that different labeling errors may have different impact, so it does not necessarily lead to the best decision. We need a principled way to evaluate a classifier and to turn scores into decisions for a given application.

BLOCCO 1 — Da accuracy a empirical Bayes risk *possibile*

domanda 4pt · uscito 18-giu-2026

Why accuracy is not enough

accuracy $= \frac{\#\text{ correct}}{\#\text{ samples}}$, **error rate** $= 1 -$ accuracy. Although sometimes useful, accuracy suffers from three issues:

- it does **not account for the cost** of the different kinds of errors;

- it depends on the **empirical class prior** of the evaluation set, which need not reflect the application prior;
- it is **unnormalized**: by itself it does not tell whether the classifier is useful (e.g. better than decisions taken without it).

We want a single-number metric that depends on the application (not on the empirical prior), accounts for error costs, allows comparing classifiers, and is normalized.

Confusion matrix and per-class error rates

For a binary task (TN, FN, FP, TP) we define:

$$P_{fn} = \frac{FN}{FN + TP}, \qquad P_{fp} = \frac{FP}{FP + TN}, \qquad TPR = 1 - P_{fn}, \qquad TNR = 1 - P_{fp}$$

Each rate uses one class at a time, hence it does **not** depend on the class proportions, only on the within-class performance. With the empirical prior $\pi_E^{emp} = \frac{TP+FN}{N}$, the error rate is a prior-weighted sum:

$$\text{err} = P_{fp}(1 - \pi_E^{emp}) + P_{fn}\,\pi_E^{emp}$$

This lets us predict the error under the **real** application prior π from per-class rates measured on a balanced set, $\text{err}_{app} = P_{fp}(1 - \pi) + P_{fn}\,\pi$, without collecting many rare-class samples.

Cost and Bayes risk

The **cost** is not monetary: it quantifies the relative effect of different errors and changes per application. With action a and cost $C(a|k)$, the **Bayes risk** is the cost we expect to pay on the application population:

$$B = E_{X,C|E}[C(a(x,R)|c)] = \sum_c \pi_c\, E_{X|C,E}[C(a(x,R)|c) \mid c]$$

Since $f_{X|C,E}$ is unknown, we approximate the expectations by averaging over a labeled evaluation set, obtaining the **empirical Bayes risk**:

$$B_{emp} = \sum_c \pi_c \frac{1}{N_c} \sum_{i:\, c_i = c} C(a(x_i, R)|c)$$

For a binary task with $C(H_F|H_F) = C(H_T|H_T) = 0$, costs C_{fn}, C_{fp}:

$$B_{emp} = \pi_T C_{fn} P_{fn} + (1 - \pi_T) C_{fp} P_{fp} \;=\; DCF_u(\pi_T, C_{fn}, C_{fp})$$

the **unnormalized Detection Cost Function**. Computing it requires the confusion matrix, the cost matrix and the class priors.

BLOCCO 2 — Da costo a metrica operativa + decisione ottima
possibile domanda 4pt · candidato fresco

Normalized DCF, dummy systems, effective prior

A **dummy system** uses only priors and costs, ignoring the data: always-accept gives $DCF_u = (1 - \pi_T) C_{fp}$, always-reject gives $DCF_u = \pi_T C_{fn}$. The **normalized DCF** divides by the best dummy:

$$DCF(\pi_T, C_{fn}, C_{fp}) = \frac{DCF_u}{\min(\pi_T C_{fn},\, (1 - \pi_T) C_{fp})}$$

A value close to 1 means no better than a dummy, close to 0 means much better. The normalized DCF is **invariant to scaling of the costs**: priors and costs can be absorbed into a single **effective prior**

$$\tilde{\pi} = \frac{\pi_T C_{fn}}{\pi_T C_{fn} + (1 - \pi_T) C_{fp}}$$

so the working point (π_T, C_{fn}, C_{fp}) is redundant (equivalent triplets give the same decision rule). For multiclass tasks no single effective prior exists, but the empirical Bayes risk and a normalized DCF (scaled by the best dummy) can still be computed.

Optimal Bayes decision

For a probabilistic classifier, the expected cost of action a is $C_{x,R}(a) = \sum_k C(a|k)P(C = k|x, R)$, and the **optimal Bayes decision** minimizes it: $a^*(x, R) = \arg\min_a C_{x,R}(a)$. If recognizer and evaluator agree ($C|X, R \sim C|X, E$), Bayes decisions minimize the Bayes risk, since $C_{x,R}(a) \geq C_{x,R}(a^*)\ \forall x$. For a generative model the rule becomes a likelihood-ratio test:

$$\log \frac{f_{X|H}(x|H_T)}{f_{X|H}(x|H_F)} \gtrless -\log \frac{\pi_T C_{fn}}{(1-\pi_T)C_{fp}}$$

For well-calibrated LLRs $s = \log \frac{f(x|H_T)}{f(x|H_F)}$ the optimal threshold is

$$t = -\log \frac{\tilde{\pi}}{1-\tilde{\pi}}$$

LLRs **disentangle the classifier from the application**: the same scores serve any working point.

BLOCCO 3 — Valutazione su tutte le soglie: curve + min/actual DCF *possibile domanda 4pt · candidato fresco · ponte verso Calibration*

ROC, DET, Equal Error Rate

Decisions compare a score with a threshold; varying the threshold yields all (P_{fn}, P_{fp}) pairs. The **Equal Error Rate** is the threshold where $P_{fn} = P_{fp}$. The **ROC** curve plots TPR vs FPR, the **DET** curve plots FNR vs FPR: both summarize performance across all thresholds.

Minimum vs actual DCF

The **actual DCF** uses the theoretical threshold $t = -\log \frac{\tilde{\pi}}{1-\tilde{\pi}}$ (assuming calibrated scores). The **minimum DCF** sweeps the threshold over the evaluation set and keeps the lowest DCF: it is the cost we would pay if we knew the optimal threshold beforehand, and measures the classifier's pure discriminative quality. The difference

$$\text{actual DCF} - \text{min DCF} \geq 0$$

is the **loss due to score mis-calibration**, which calibration techniques aim to reduce.

[INTEGRAZIONE TEORICA]
Il **min DCF** isola il potere discriminante: *quanto bene* il modello separa le classi, indipendentemente dalla soglia. L'**actual DCF** include anche *quanto bene* gli score sono calibrati per quella specifica applicazione. Per questo il gap tra i due è la firma della mis-calibrazione, e non del potere discriminante: due modelli con lo stesso min DCF possono avere actual DCF molto diversi.

15 Score Calibration and Fusion

Motivation. For a given application the gap between actual and minimum DCF is the loss due to **score mis-calibration**. Calibration techniques aim to close this gap; fusion combines several classifiers into a single, better decision.

BLOCCO A — Calibrazione (cuore della domanda) *possibile*
domanda 4pt · candidato fresco

Why scores are mis-calibrated

In many cases classifier scores have **no clear probabilistic interpretation** and are not log-likelihood ratios for the evaluation population:

- **SVM** scores cannot be interpreted as probabilities;
- strongly **regularized logistic regression** distorts scores so they no longer reflect class probabilities;
- even **generative models** produce poorly calibrated scores under overfitting or a **train/test mismatch**.

Deciding directly on such scores leads to mis-calibration, i.e. actual DCF > min DCF.

Two strategies

To reduce mis-calibration we can: (1) use a **validation set to find a (near-)optimal threshold** for a given application — but we must know the target application; or (2) learn a **transformation f that maps scores into approximately well-calibrated LLRs**, so optimal decisions are obtained for different applications (**application-independent**). We follow (2). We require f **monotone**, so that higher raw scores keep favoring H_T and lower ones H_F: $s_{cal} = f(s)$.

Three calibration methods

- **Isotonic regression:** non-parametric, monotone, optimal fit on the training data. The function is **piecewise** non-linear, needs interpolation for unseen scores, allows **no extrapolation**, and is expensive for large calibration sets.
- **Prior-weighted logistic regression:** an **affine** mapping incorporating class priors; simple, **extrapolates**, fast, but good only over a limited range of operating points.
- **Generative score models:** assume a probabilistic model for the per-class score distribution.

Prior-weighted logistic regression calibration

Raw scores are treated as 1-D feature vectors with an affine map $f(s) = \alpha s + \gamma$. Since $f(s)$ should be an LLR:

$$f(s) = \log \frac{f_{S|C}(s|H_T)}{f_{S|C}(s|H_F)} = \alpha s + \gamma$$

The posterior log-odds for prior $\tilde{\pi}$ are

$$\log \frac{P(C = H_T|s)}{P(C = H_F|s)} = \alpha s + \gamma + \log \frac{\tilde{\pi}}{1 - \tilde{\pi}} = \alpha s + \beta$$

We estimate α, β with a non-regularized **prior-weighted** logistic regression on a calibration training set, then the calibrated score is

$$f(s) = \alpha s + \beta - \log \frac{\tilde{\pi}}{1-\tilde{\pi}}$$

with objective ($w_i = \tilde{\pi}/n_T$ if $z_i = +1$, $(1-\tilde{\pi})/n_F$ if $z_i = -1$):

$$R(\alpha, \gamma) = \sum_i w_i \log\Big(1 + e^{-z_i\,(\alpha s + \gamma + \log \frac{\tilde{\pi}}{1-\tilde{\pi}})}\Big)$$

We must specify a prior $\tilde{\pi}$, so we optimize for a specific application, but calibration is usually good for nearby applications too.

Generative score model

Alternatively, model the class-conditional score densities and use their LLR:

$$f_{cal}(s) = \log \frac{f_{S|H_T}(s)}{f_{S|H_F}(s)}, \qquad f_{S|H_T} = \mathcal{N}(s|\mu_T, v_T),\ f_{S|H_F} = \mathcal{N}(s|\mu_F, v_F)$$

Tied variances $v_T = v_F = v$ are used because they yield a monotone transformation.

BLOCCO B — Setup dei dati + fusion *più lato-progetto che teoria 4pt*

Calibration set, data splits, K-fold

The calibration set must be **independent** from the model-training and validation/evaluation sets, otherwise results are biased. The pipeline splits the training material into three parts: **Model train | Calibration train | (Calibration) Validation**; for convenience, calibration and validation are often extracted from the former validation set, reusing already-trained models. Two scenarios:

- mis-calibration from non-probabilistic scores or over/under-fitting, with **similar** populations ⇒ calibration set from the training material; being 1-D, overfitting risk is low and no regularization is needed;
- mis-calibration from a **mismatch** between training and application populations ⇒ the calibration set must **mimic the application** (less data than training a full classifier; Gaussian models can exploit unlabeled data).

With little data we use **K-fold cross-validation**, where every sample plays each role. The metric must be computed on the **pooled scores** of all folds (not by averaging per-fold metrics), using the same threshold; **LOO** is the extreme $K = N$. The final model is retrained on the **whole** training set, since each fold model used only a fraction $\frac{K-1}{K}$ of the data.

Score fusion

Combining classifiers that extract different information. **Majority voting** assigns the most-voted label but needs tie-breaking and ignores confidence. With scores, a **weighted fusion** is used:

$$s_{fused} = \alpha^T s + \gamma$$

an affine transformation of the score vector **s** of the m classifiers; weights α and bias γ are estimated with **logistic regression**, exactly as in calibration but on m-dimensional inputs (summing LLRs is correct only if the systems are independent). With a single system this reduces to calibration. For multiclass tasks, multiclass logistic regression estimates the calibration/fusion weights.